Estrutura Básica de um Aplicativo Flutter (12H)

Widgets

Conceito de Widgets:

Widgets no Flutter são elementos fundamentais que compõem a interface do usuário. Eles são componentes autônomos e reutilizáveis, cada um representando uma parte específica da interface gráfica. Pode-se pensar em widgets como peças de LEGO que, quando combinadas, formam a estrutura visual do aplicativo. Existem widgets para diferentes propósitos, desde exibir texto simples até criar layouts mais complexos e interativos.

Árvore de Widgets:

No Flutter, os widgets são organizados hierarquicamente em uma estrutura de árvore. Cada aplicativo Flutter possui um widget principal chamado MaterialApp ou CupertinoApp (dependendo do design de Android ou iOS), que serve como a raiz da árvore de widgets. Dentro dessa árvore, cada widget tem um pai e zero ou mais filhos.

Pai e Filhos:

- Pai: É o widget que contém outros widgets.
- Filhos: São os widgets contidos dentro de um widget pai.

Hierarquia:

Widgets na parte inferior da árvore são geralmente widgets de alto nível, como páginas ou layouts.
À medida que você sobe na árvore, os widgets tornam-se mais específicos e detalhados.

Widgets de Estrutura no Flutter

Os Widgets de Estrutura são aqueles que servem como base para a construção da interface do usuário. Eles fornecem o layout principal de uma tela e ajudam a organizar os elementos da UI de maneira estruturada.

Scaffold - O Container Principal de uma Página

O Scaffold é um dos widgets mais fundamentais do Flutter. Ele fornece uma estrutura básica para construir uma tela e já vem com componentes embutidos, como:

- AppBar (barra superior de navegação)
- Body (corpo da tela)
- FloatingActionButton (botão flutuante)
- Drawer (menu lateral)
- BottomNavigationBar (barra de navegação inferior)
- SnackBar (notificações curtas)

Exemplo de Uso do Scaffold

```
import 'package:flutter/material.dart';

void main() {
  runApp(MyApp());
}

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        appBar: AppBar(
          title: Text('Exemplo de Scaffold'),
          backgroundColor: Colors.blue,
        ),
        body: Center(
          child: Text('Conteúdo da tela', style: TextStyle(fontSize: 20)),
        ),
        floatingActionButton: FloatingActionButton(
          onPressed: () {
            print('Botão pressionado!');
          },
          child: Icon(Icons.add),
        ),
```

```
      bottomNavigationBar: BottomNavigationBar(
        items: [
          BottomNavigationBarItem(icon: Icon(Icons.home), label: 'Home'),
          BottomNavigationBarItem(icon: Icon(Icons.person), label: 'Perfil'),
        ],
      ),
    ),
  );
 }
}
```

Explicação dos Componentes do Scaffold

- appBar: Adiciona uma barra superior à tela.
- body: Define o corpo principal da interface.
- floatingActionButton: Um botão de ação flutuante, geralmente usado para ações principais.
- bottomNavigationBar: Barra de navegação inferior para trocar de telas.

AppBar - Barra de Aplicação

A AppBar é um widget que fornece uma barra superior de navegação dentro do Scaffold.

Exemplo de AppBar Personalizada

```
AppBar(
  title: Text('Minha Aplicação'),
  backgroundColor: Colors.deepPurple,
  actions: [
    IconButton(
      icon: Icon(Icons.search),
      onPressed: () {
        print('Clicou na lupa');
      },
    ),
    IconButton(
      icon: Icon(Icons.notifications),
      onPressed: () {
        print('Clicou no sino');
      },
```

```
    ),
  ],
)
```

O que foi adicionado?

- Um título na barra superior.
- Ícones de pesquisa e notificações com ações definidas.

Drawer - Menu Lateral

O Drawer é um menu lateral deslizante, geralmente usado para navegação entre diferentes telas. Ele é ativado ao tocar no ícone de menu (☰) na AppBar.

Exemplo de Drawer com Itens de Menu

```
Scaffold(

  appBar: AppBar(title: Text('Exemplo de Drawer')),
  drawer: Drawer(
    child: ListView(
      children: [
        DrawerHeader(
          decoration: BoxDecoration(color: Colors.blue),
          child: Text('Menu',
                                    style: TextStyle(
                                              color: Colors.white,
                                                        fontSize: 24)),
                              ),
        ListTile(
          leading: Icon(Icons.home),
          title: Text('Início'),
          onTap: () {
            print('Início pressionado');
          },
        ),
        ListTile(
          leading: Icon(Icons.settings),
          title: Text('Configurações'),
          onTap: () {
            print('Configurações pressionado');
```

```
            },
          ),
        ],
      ),
    ),
  )
```

Explicação:

- DrawerHeader: Define um cabeçalho estilizado.
- ListTile: Cria itens do menu, cada um com um ícone e um texto.

BottomNavigationBar - Barra Inferior de Navegação

A BottomNavigationBar é usada para criar abas de navegação na parte inferior do aplicativo.

Exemplo de BottomNavigationBar

```
  Scaffold(

    bottomNavigationBar: BottomNavigationBar(
      items: [
        BottomNavigationBarItem(icon: Icon(Icons.home), label: 'Home'),
        BottomNavigationBarItem(icon: Icon(Icons.search), label: 'Buscar'),
        BottomNavigationBarItem(icon: Icon(Icons.person), label: 'Perfil'),
      ],
    ),
  )
```

Explicação:

BottomNavigationBarItem: Define um ícone e um rótulo para cada aba.

FloatingActionButton - Botão de Ação Flutuante

O FloatingActionButton (FAB) é um botão flutuante circular usado para a principal ação da tela.

Exemplo de FloatingActionButton

```
Scaffold(

  floatingActionButton: FloatingActionButton(
    onPressed: () {
      print('Ação do botão flutuante');
    },
    child: Icon(Icons.add),
  ),
)
```

Explicação:

- onPressed: Define a ação quando o botão é pressionado.
- child: Define o conteúdo do botão (ícone ou texto).

SnackBar - Mensagens Temporárias

O SnackBar exibe notificações temporárias na parte inferior da tela.

Exemplo de SnackBar

```
Scaffold(
  body: Center(
    child: ElevatedButton(
      onPressed: () {
        ScaffoldMessenger.of(context).showSnackBar(
          SnackBar(content: Text('Ação realizada com sucesso!')),
        );
      },
      child: Text('Mostrar SnackBar'),
    ),
  ),

)
```

Explicação:

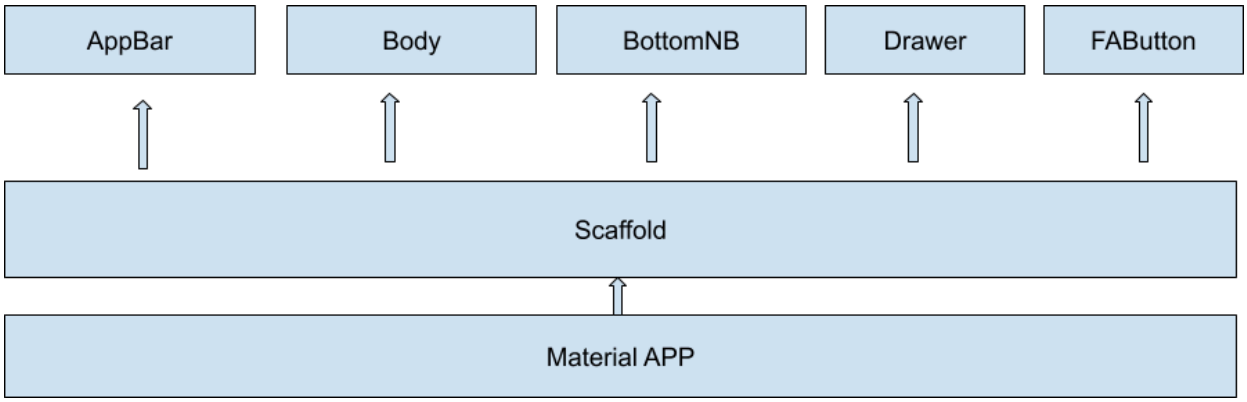
ScaffoldMessenger.of(context).showSnackBar(): Exibe uma mensagem temporária.

Conclusão

Os Widgets de Estrutura fornecem a base para criar telas bem organizadas no Flutter. Com eles, podemos definir menus, navegação, botões e mensagens para interações no app.

Resumo:

Widget	Função Principal
Scaffold	Estrutura base da tela
AppBar	Barra superior com título e ações
Drawer	Menu lateral deslizante
BottomNavigationBar	Barra de navegação inferior
FloatingActionButton	Botão flutuante para ação principal
SnackBar	Notificações temporárias



Widgets de Layout no Flutter

Os Widgets de Layout são responsáveis por definir como os elementos são organizados e posicionados na tela. Eles ajudam a estruturar a interface do usuário, controlando o alinhamento, espaçamento e distribuição dos componentes.

Column - Layout Vertical

O widget Column organiza os elementos verticalmente, ou seja, um abaixo do outro.

Propriedades Principais do Column:

- mainAxisAlignment – Controla o alinhamento dos widgets no eixo principal (vertical).
- crossAxisAlignment – Controla o alinhamento no eixo cruzado (horizontal).
- children – Lista de widgets dentro da coluna.

Exemplo de Column

```
Column(

  mainAxisAlignment: MainAxisAlignment.center, // Alinha os elementos ao centro
verticalmente
  crossAxisAlignment: CrossAxisAlignment.center, // Alinha ao centro horizontalmente
  children: [
    Text('Linha 1', style: TextStyle(fontSize: 20)),
    Text('Linha 2', style: TextStyle(fontSize: 20)),
  ],
```

```
)
```

Explicação:

- mainAxisAlignment.center → Centraliza os widgets no meio do eixo vertical.
- crossAxisAlignment.center → Alinha os widgets no centro horizontal.

Outras opções para mainAxisAlignment:

- start: Alinha os elementos no topo.
- end: Alinha os elementos na parte inferior.
- spaceBetween: Distribui os widgets com espaçamento igual entre eles.
- spaceEvenly: Distribui os widgets uniformemente na tela.

Row - Layout Horizontal

O widget Row organiza os elementos horizontalmente, ou seja, lado a lado.

Propriedades Principais do Row:

- mainAxisAlignment – Controla o alinhamento no eixo principal (horizontal).
- crossAxisAlignment – Controla o alinhamento no eixo cruzado (vertical).
- children – Lista de widgets dentro da linha.

Exemplo de Row

```
Row(
  mainAxisAlignment: MainAxisAlignment.spaceAround, // Distribui os widgets
uniformemente
  crossAxisAlignment: CrossAxisAlignment.center, // Alinha os widgets ao centro
verticalmente
  children: [
    Icon(Icons.star, size: 40, color: Colors.blue),
    Icon(Icons.favorite, size: 40, color: Colors.red),
    Icon(Icons.home, size: 40, color: Colors.green),
  ],
)
```

Explicação:

- mainAxisAlignment.spaceAround → Distribui os elementos com espaços iguais ao redor.

- crossAxisAlignment.center → Mantém os elementos centralizados verticalmente.

Outras opções para mainAxisAlignment:

- start: Alinha os elementos à esquerda.
- end: Alinha os elementos à direita.
- spaceBetween: Coloca o primeiro e o último widget nas extremidades e distribui os outros no meio.
- spaceEvenly: Distribui os widgets uniformemente na linha.

Stack - Sobreposição de Widgets

O widget Stack permite que os widgets sejam posicionados uns sobre os outros, criando layouts mais complexos.

Propriedades do Stack:

- alignment – Define o alinhamento dos elementos dentro da pilha.
- children – Lista de widgets que serão empilhados.
- Positioned – Define a posição específica de um elemento dentro do Stack.

Exemplo de Stack

```
Stack(
  alignment: Alignment.center, // Centraliza os elementos na pilha
  children: [
    Container(
      width: 200,
      height: 200,
      color: Colors.blue,
    ),
    Text('Texto Sobreposto', style: TextStyle(fontSize: 20, color: Colors.white)),
  ],
)
```

Explicação:

- O Container azul é a base do Stack.
- O Texto está sobreposto ao Container, centralizado.

Usando Positioned para definir posições específicas:

```
Stack(
  children: [
    Container(width: 200, height: 200, color: Colors.blue),
    Positioned(
      top: 20,
      right: 20,
      child: Icon(Icons.star, size: 40, color: Colors.yellow),
    ),
  ],
)
```

O que acontece aqui?

- O ícone da estrela aparece no canto superior direito do container azul.

Container - Controle de Tamanho, Bordas e Margens

O Container é um dos widgets mais versáteis do Flutter. Ele permite definir tamanho, espaçamentos, cores e bordas para organizar a UI.

Propriedades do Container:

- width e height – Controlam a largura e altura.
- color – Define a cor de fundo.
- margin – Define o espaçamento externo.
- padding – Define o espaçamento interno.
- decoration – Adiciona bordas, sombras e personalizações.

Exemplo de Container

```
Container(
  width: 200,
  height: 100,
  margin: EdgeInsets.all(20), // Margem externa de 20 pixels
  padding: EdgeInsets.all(10), // Espaçamento interno de 10 pixels
  decoration: BoxDecoration(
    color: Colors.orange,
    borderRadius: BorderRadius.circular(10), // Bordas arredondadas
    boxShadow: [
      BoxShadow(
        color: Colors.black26,
```

```
        blurRadius: 5, // Efeito de sombra
        spreadRadius: 2,
      ),
    ],
  ),
  child: Center(
    child: Text(
      'Texto dentro do Container',
      style: TextStyle(color: Colors.white, fontSize: 16),
    ),
  ),
)
```

Explicação:

- O Container tem uma cor de fundo laranja.
- Ele possui bordas arredondadas e sombra para dar um efeito elevado.
- Usa margin para espaçamento externo e padding para espaçamento interno.

Resumo dos Widgets de Layout

Widget	Descrição
Column	Organiza elementos verticalmente.
Row	Organiza elementos horizontalmente.
Stack	Sobrepõe elementos, permitindo layouts complexos.

Container	Define tamanho, cor, bordas e espaçamentos.

Widgets de Exibição no Flutter

Os Widgets de Exibição são responsáveis por exibir informações na tela do aplicativo. Eles incluem texto, imagens e ícones, possibilitando a criação de interfaces ricas e informativas.

Text - Exibição de Texto

O Text é um dos widgets mais utilizados no Flutter. Ele permite exibir mensagens, títulos, descrições e qualquer outro conteúdo textual.

Propriedades Principais:

- style – Define a aparência do texto (tamanho, cor, negrito, itálico etc.).
- textAlign – Alinhamento do texto (left, center, right).
- overflow – Define como lidar com textos longos (ellipsis, clip, fade).
- maxLines – Define o número máximo de linhas exibidas.

Exemplo de Uso Simples

```
Text(
  'Olá, Flutter!',
  style: TextStyle(fontSize: 20, color: Colors.blue),
)
```

Exemplo com Personalização

```
Text(
  'Flutter é incrível!',
```

```
  textAlign: TextAlign.center, // Centraliza o texto
  style: TextStyle(
    fontSize: 24, // Tamanho do texto
    fontWeight: FontWeight.bold, // Texto em negrito
    color: Colors.purple, // Cor do texto
    letterSpacing: 2, // Espaçamento entre letras
  ),
)
```

Image - Exibição de Imagens

O Image é usado para carregar e exibir imagens de diferentes fontes:

- Image.asset() → Imagens locais (armazenadas na pasta do projeto).
- Image.network() → Imagens da internet.
- Image.file() → Imagens armazenadas no dispositivo do usuário.

Exemplo de Imagem da Internet

```
Image.network(
  'https://storage.googleapis.com/cms-storage-bucket/9abb63d8732b978c7ea1.png',
  width: 200, // Define a largura
  height: 100, // Define a altura
  fit: BoxFit.cover, // Ajusta a imagem dentro do espaço
)
```

Exemplo de Imagem Local (Armazenada no Projeto)

Adicione a imagem na pasta assets/images/
No arquivo pubspec.yaml, registre o caminho:

```
flutter:
  assets:
    - assets/images/flutter_logo.png
```

Agora, exiba a imagem com Image.asset()

```
Image.asset(
  'assets/images/flutter_logo.png',
  width: 150,
  height: 150,
)
```

Explicação:

- fit: BoxFit.cover → Ajusta a imagem para preencher o espaço disponível.
- width e height → Define o tamanho da imagem.

Icon - Exibição de Ícones

O Icon permite exibir ícones do Material Design. É útil para botões, menus e indicadores visuais.

Propriedades Principais:

Icons.nome_do_ícone → Define o ícone a ser exibido.
size → Define o tamanho do ícone.
color → Define a cor do ícone.

Exemplo Simples de Ícone

```
Icon(
```

```
  Icons.star, // Ícone de estrela
  color: Colors.amber, // Cor amarela
  size: 30, // Tamanho do ícone
)
```

Exemplo de Ícone dentro de um Botão

```
ElevatedButton.icon(
  onPressed: () {},
  icon: Icon(Icons.thumb_up, color: Colors.white),
  label: Text('Curtir'),
  style: ElevatedButton.styleFrom(backgroundColor: Colors.blue),
)
```

Resumo dos Widgets de Exibição

Widget	Descrição
Text	Exibe textos personalizados.
Image	Exibe imagens locais ou da internet.
Icon	Exibe ícones do Material Design.

Exercício Prático

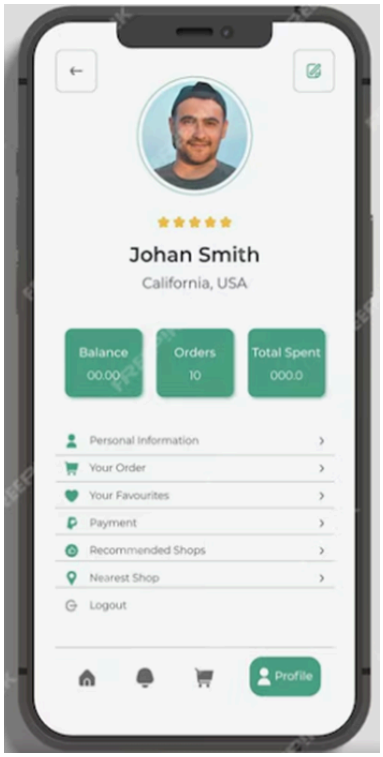
Crie uma tela de perfil básica usando Column, Row, Stack e Container.

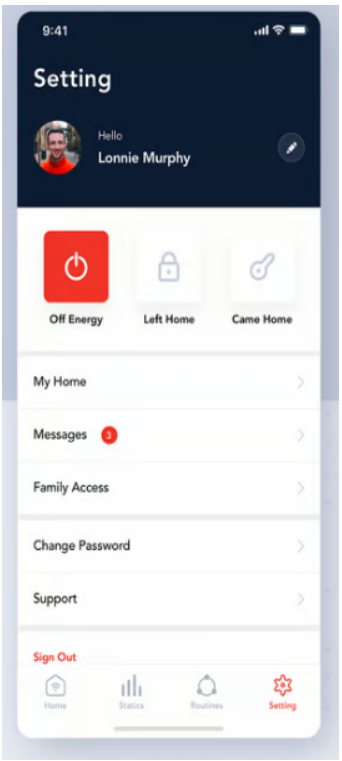
Requisitos:

Exibir uma imagem de perfil.

- Exibir nome e descrição abaixo da imagem.
- Criar uma linha com ícones de redes sociais.

  Exemplos:





AppBar -> Icones

Container -> Imagem

Nome

Outra Info Pessoal

Façam 3 Containers(coloridos) em uma Linha

Lista de Texto (5 pelo menos)

BottonNavigationBar -> icones (3 icones)

SA01 - Usando um GridView para Exibir Imagens Junto com um Carrossel de Imagens

---

Stateless vs Stateful Widgets no Flutter

No Flutter, os widgets são a base da interface do usuário e são divididos em Stateless e Stateful widgets, dependendo se precisam ou não armazenar e atualizar estados.

Stateless Widgets (Widgets Sem Estado)

Os Stateless Widgets são imutáveis, ou seja, não podem mudar após serem criados.
Eles são ideais para exibir dados estáticos que não precisam ser alterados durante a execução do aplicativo.

Características Principais:

- Não armazenam estado interno.
- Construídos apenas uma vez e não são reconstruídos, a menos que um novo widget os substitua.
- Perfeitos para layouts simples e conteúdos fixos.

Exemplo de StatelessWidget

```
import 'package:flutter/material.dart';

class MeuWidget extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Center(
```

```
      child: Text(
        'Eu sou um StatelessWidget!',
        style: TextStyle(fontSize: 20, color: Colors.blue),
      ),
    );
  }
}

void main() {
  runApp(MaterialApp(
    home: Scaffold(
      appBar: AppBar(title: Text('Stateless Widget')),
      body: MeuWidget(),
    ),
  ));
}
```

Explicação:

- O widget MeuWidget apenas exibe um texto fixo.
- Como ele não precisa armazenar ou modificar estado, é um StatelessWidget.

Stateful Widgets (Widgets com Estado)

Os Stateful Widgets podem mudar dinamicamente enquanto o aplicativo está em execução. Eles são ideais para interações do usuário, atualizações de dados e animações.

Características Principais:

- Podem armazenar e atualizar estados internos.
- Reconstroem a interface sempre que o estado muda.
- Usados para formulários, contadores, animações e elementos dinâmicos.

Estrutura de um StatefulWidget

```
import 'package:flutter/material.dart';

class MeuWidget extends StatefulWidget {
  @override
  _MeuWidgetState createState() => _MeuWidgetState();
```

```
}

class _MeuWidgetState extends State<MeuWidget> {
  int contador = 0; // Estado da classe

  @override
  Widget build(BuildContext context) {
    return Column(
      mainAxisAlignment: MainAxisAlignment.center,
      children: [
        Text(
          'Contador: $contador',
          style: TextStyle(fontSize: 24),
        ),
        SizedBox(height: 20),
        ElevatedButton(
          onPressed: () {
            setState(() {
              contador++; // Atualiza o estado
            });
          },
          child: Text('Incrementar'),
        ),
      ],
    );
  }
}

void main() {
  runApp(MaterialApp(
    home: Scaffold(
      appBar: AppBar(title: Text('Stateful Widget')),
      body: Center(child: MeuWidget()),
    ),
  ));
}
```

Explicação:

- Criamos um widget chamado MeuWidget, que contém um contador.
- A variável contador armazena o estado do widget.
- O método setState() atualiza a interface sempre que o botão for pressionado.

Comparação Entre Stateless e Stateful

Característica	StatelessWidget	StatefulWidget
Mutabilidade	Imutável (não muda)	Mutável (pode mudar)
Uso ideal	Layouts fixos e exibição de dados estáticos	Interação do usuário, animações e dados dinâmicos
Armazena estado?	Não	Sim
Método principal	build()	build() + setState()

Exemplo Prático - Mudando de Stateless para Stateful

Vamos ver um exemplo prático, onde transformamos um StatelessWidget em StatefulWidget.

Cenário:

Queremos criar um botão que muda a cor do texto ao ser pressionado.

Versão com StatelessWidget (Não Funciona!)

```
import 'package:flutter/material.dart';

class MeuWidget extends StatelessWidget {
  Color corTexto = Colors.black; // Não pode ser alterado!

  @override
  Widget build(BuildContext context) {
    return Column(
      children: [
        Text(
          'Texto colorido!',
          style: TextStyle(fontSize: 24, color: corTexto),
        ),
        ElevatedButton(
          onPressed: () {
            corTexto = Colors.red; // ❌ ERRO! Não pode alterar estado em StatelessWidget
```

```
        },
        child: Text('Mudar Cor'),
      ),
    ],
  );
 }
}

void main() {
 runApp(MaterialApp(home: Scaffold(body: MeuWidget())));
}
```

Por que isso não funciona?

Porque StatelessWidgets são imutáveis e não permitem mudanças no estado.

Versão Correta com StatefulWidget

```
import 'package:flutter/material.dart';

class MeuWidget extends StatefulWidget {
 @override
 _MeuWidgetState createState() => _MeuWidgetState();
}

class _MeuWidgetState extends State<MeuWidget> {
 Color corTexto = Colors.black; // Estado inicial

 void mudarCor() {
  setState(() {
   corTexto = corTexto == Colors.black ? Colors.red : Colors.black;
  });
 }

 @override
 Widget build(BuildContext context) {
  return Column(
   mainAxisAlignment: MainAxisAlignment.center,
   children: [
    Text(
     'Texto colorido!',
```

```
        style: TextStyle(fontSize: 24, color: corTexto),
      ),
      SizedBox(height: 20),
      ElevatedButton(
        onPressed: mudarCor,
        child: Text('Mudar Cor'),
      ),
    ],
  );
 }
}

void main() {
 runApp(MaterialApp(home: Scaffold(body: Center(child: MeuWidget()))));
}
```

Agora funciona porque usamos setState(), que atualiza a interface!

Conclusão

Quando usar StatelessWidget?

- Para exibir dados estáticos (textos, imagens, layouts fixos).
- Quando não há necessidade de atualização dinâmica.
- Exemplo: Um título, um logotipo, um ícone fixo.

Quando usar StatefulWidget?

- Quando o conteúdo precisa mudar durante a execução do app.
- Para interação do usuário (botões, animações, campos de formulário).
- Exemplo: Contadores, listas dinâmicas, botões de "Curtir", temas dinâmicos.

Dica: Sempre comece com StatelessWidget. Se perceber que precisa de mudanças dinâmicas, transforme-o em StatefulWidget!

Situação de Aprendizagem 2 - Construção de um Aplicativo de Lista de Tarefas em Flutter

Objetivo

Desenvolver um aplicativo Flutter de Lista de Tarefas (To-Do List), utilizando widgets básicos e interativos, permitindo que o usuário adicione, visualize e remova tarefas.

Desafio Proposto

Você foi contratado para criar um gerenciador de tarefas pessoais. O aplicativo precisa ter:

- Uma tela principal listando as tarefas.
  Um campo para adicionar novas tarefas.
  Um botão para remover tarefas concluídas.
  Feedback visual ao concluir uma tarefa.

Etapas do Desenvolvimento

Estrutura do Projeto

O app terá apenas uma tela principal, dividida em:

- Campo de entrada para adicionar tarefas.
- Lista de tarefas usando ListView.
- Botão para excluir tarefas concluídas.

Protótipo Visual

```
-----------------------------------------------
|        📝 Tarefas                           |
|---------------------------------------------|
|  [Digite uma tarefa...] (Campo)       |
|  [ ADICIONAR ] (Botão)                |
|---------------------------------------------|
|  ✅ Comprar pão [X]                   |
|  ⭕ Estudar Flutter [X]               |
|  ⭕ Fazer exercícios [X]              |
|---------------------------------------------|
```

| [REMOVER CONCLUÍDAS] (Botão)|

Desenvolvimento do Código

https://docs.google.com/document/d/15AAR4XZtfrNUh-9-z5jZzuB7DJM4KPTI3QbPtuFm0l0/edit?tab=t.syy9hakhg3lq

---

Widgets Interativos no Flutter

Os Widgets Interativos permitem que o usuário interaja com a interface da aplicação. Eles incluem botões, campos de entrada, caixas de seleção e sliders, possibilitando a captura e manipulação de dados.

ElevatedButton - Botão Elevado

O ElevatedButton é um botão com elevação, que destaca sua presença na tela. Ele é amplamente utilizado para ações primárias, como submissões e confirmações.

Propriedades Principais:

- onPressed – Define a ação ao pressionar o botão.
- child – Define o conteúdo do botão (texto, ícone, etc.).
- style – Personaliza a aparência do botão.

Exemplo de ElevatedButton Simples

```
ElevatedButton(
  onPressed: () {
    print('Botão pressionado');
  },
  child: Text('Clique aqui'),
)
```

Exemplo de ElevatedButton Personalizado

```
ElevatedButton(
  onPressed: () {
    print('Botão personalizado pressionado');
  },
  style: ElevatedButton.styleFrom(
    backgroundColor: Colors.blue, // Cor de fundo
    padding: EdgeInsets.symmetric(horizontal: 20, vertical: 12), // Espaçamento interno
    shape: RoundedRectangleBorder(
      borderRadius: BorderRadius.circular(10), // Bordas arredondadas
    ),
  ),
  child: Text(
    'Botão Customizado',
    style: TextStyle(fontSize: 18, color: Colors.white),
  ),
)
```

Explicação:

- backgroundColor → Define a cor de fundo.
- padding → Ajusta o espaço interno.
- borderRadius → Arredonda as bordas.

TextField - Campo de Entrada de Texto

O TextField permite que o usuário insira texto, como em formulários, login e busca.

Propriedades Principais:

- controller – Obtém o valor digitado.
- decoration – Personaliza a aparência.
- obscureText – Oculta o texto (útil para senhas).

Exemplo de TextField Simples

```
TextField(
  decoration: InputDecoration(
    labelText: 'Digite seu nome',
    border: OutlineInputBorder(),
  ),
)
```

Exemplo de TextField com Controller

```
TextEditingController nomeController = TextEditingController();

TextField(
  controller: nomeController,
  decoration: InputDecoration(
    labelText: 'Nome',
    hintText: 'Digite seu nome completo',
    border: OutlineInputBorder(),
    prefixIcon: Icon(Icons.person),
  ),
)
```

Para acessar o valor digitado:

```
print('Nome digitado: ${nomeController.text}');
```

Checkbox - Caixa de Seleção

O Checkbox permite selecionar ou desmarcar uma opção, útil em listas de opções ou configurações.

Propriedades Principais:

- value – Define se está marcado (true) ou não (false).
- onChanged – Dispara uma ação ao mudar o estado.

Exemplo de Checkbox

```
bool isChecked = false;

Checkbox(
  value: isChecked,
  onChanged: (bool? newValue) {
    isChecked = newValue!;
    print('Checkbox selecionado: $isChecked');
  },
)
```

Versão com Texto ao Lado:

```
Row(
  children: [
    Checkbox(
      value: isChecked,
      onChanged: (bool? newValue) {
        isChecked = newValue!;
      },
    ),
    Text('Aceito os termos e condições'),
  ],
)
```

Slider - Controle Deslizante

O Slider permite selecionar um valor dentro de um intervalo, sendo útil para ajustes como volume, brilho e preço.

Propriedades Principais:

- value – Define o valor atual.
- min e max – Define o intervalo permitido.
- onChanged – Atualiza o valor ao deslizar.

Exemplo de Slider

```
double sliderValue = 50.0;

Slider(
  value: sliderValue,
  min: 0,
  max: 100,
  divisions: 10, // Divide em 10 partes
  label: sliderValue.round().toString(), // Exibe o valor atual
  onChanged: (double newValue) {
    sliderValue = newValue;
    print('Valor do slider: $sliderValue');
  },
)
```

Explicação:

- min e max → Define os valores mínimos e máximos.

- divisions → Divide o slider em partes iguais.
- label → Exibe o valor atual acima do cursor.

Exemplo Prático - Formulário Interativo

Vamos combinar os widgets interativos para criar um formulário com

- Campo de nome (TextField).
- Checkbox de aceite de termos.
- Slider para selecionar idade.
- Botão para enviar os dados.

Resumo dos Widgets Interativos

Widget	Descrição
ElevatedButton	Botão com elevação.
TextField	Campo para entrada de texto.
Checkbox	Caixa de seleção.
Slider	Controle deslizante para valores.

https://docs.google.com/document/d/15AAR4XZtfrNUh-9-z5jZzuB7DJM4KPTI3QbPtuFm0l0/edit?tab=t.yw05ywmpgsva

---

Situação de Aprendizagem Guiada: Estrutura de Aplicativos em Flutter

Contextualização

Imagine que você foi contratado para desenvolver um aplicativo de cadastro de usuários para uma pequena empresa. Esse aplicativo precisa ter uma interface intuitiva e responsiva, utilizando os principais widgets do Flutter.

Seu objetivo é estruturar o app com uma tela de boas-vindas, um formulário de cadastro e uma tela de confirmação após o envio do formulário.

Etapas da Atividade

Compreendendo a Estrutura do Flutter

Antes de iniciar o desenvolvimento, entenda como um aplicativo Flutter é organizado:

- main.dart → Arquivo principal que inicia o app.
- Widgets principais: Scaffold, Column, Row, Container, Stack.
- Widgets interativos: TextField, Checkbox, Slider, ElevatedButton.
- Navegação entre telas usando Navigator.

Estruturando o Aplicativo

O app terá três telas principais:

Tela 1: Boas-Vindas

- Mensagem de apresentação do app.
- Botão "Iniciar Cadastro" para ir à tela de formulário.

Tela 2: Formulário de Cadastro

- Campo para nome (TextField).
- Campo para e-mail (TextField).
- Seletor de idade (Slider).
- Checkbox para aceitar os termos (Checkbox).
- Botão para enviar os dados (ElevatedButton).

Tela 3: Confirmação

- Mensagem confirmando o cadastro.
- Botão para voltar à tela inicial.

Protótipo Visual

Abaixo está um esboço das telas para ajudar na organização do layout.

Tela de Boas-Vindas

```
-----------------------------------------------
|   📱 APP LOGO                               |
|---------------------------------------------|
|  Bem-vindo ao nosso app!                    |
|  Faça seu cadastro agora.                   |
|                                             |
|  [ INICIAR CADASTRO ] (BOTÃO)               |
-----------------------------------------------
```

Tela de Cadastro

```
-----------------------------------------------
|        📝 Formulário                        |
|---------------------------------------------|
| Nome:  ____________________                 |
| E-mail: ___________________                 |
| Idade:  [----SLIDER----] 25                 |
```

| [✔] Aceito os termos |

| |

| [ENVIAR CADASTRO] |

Tela de Confirmação

| ✅ Cadastro Feito! |

|---|

| Seu cadastro foi realizado |

| com sucesso! |

| |

| [VOLTAR AO INÍCIO] |

Desenvolvimento Passo a Passo